IT STUDY HUB

COMPLETE R PROGRAMMING SERIES

MODULE 5: DATA IMPORT, CLEANING & BASIC MANIPULATION

Authors:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

WELCOME TO MODULE 5

About This Module

Welcome to Module 5 of the **IT Study Hub Complete R Programming Series**. In the previous modules, you learned to construct data structures, design logic gates, and write custom functions. Now, we bridge the gap between academic theory and real-world data science.

In the real world, data rarely originates inside R. It lives in messy Excel sheets, massive CSV files, and cryptic text exports. Furthermore, real data is notoriously dirty—plagued with missing values, duplicate entries, and terrible column names. This module introduces the `tidyverse` ecosystem, specifically the `readr`, `readxl`, `tidyr`, and `dplyr` packages. You will learn the industry-standard workflow for ingesting raw data, surgically cleaning it, reshaping it into a 'Tidy' format, and preparing it for advanced visualization and statistical modeling.

IT STUDY HUB

Learning Outcomes

By the completion of this comprehensive module, you will be able to:

- Seamlessly import external files (CSV, Excel, Text) into the R environment.
- Identify, quantify, and strategically manage missing data (`NA`) using Dropping or Imputation.
- Execute core data cleaning operations: removing duplicates and renaming columns.
- Master the essential `dplyr` verbs: `select()`, `filter()`, `arrange()`, and `mutate()`.
- Chain complex operations together fluidly using the pipe operator (`%>%`).
- Export transformed, production-ready datasets back to the local hard drive.
- Apply the philosophical rules of 'Tidy Data' to format tables perfectly for machine learning.

Module Coverage

This handbook thoroughly covers the following core topics:

1. Reading CSV files
2. Reading Excel files
3. Importing text files
4. Exporting data
5. Understanding missing values
6. Detecting and handling NA
7. Removing duplicates
8. Renaming columns
9. Selecting columns
10. Filtering rows
11. Sorting data
12. Creating new columns
13. Basic data cleaning
14. Tidy data concepts
15. The dplyr workflow

IT STUDY HUB

MAIN NOTES

1. Reading CSV Files (readr)

CONCEPT:
CSV (Comma-Separated Values) is the most common format for raw data. R reads these files natively or via the optimized 'readr' package.

Deep Dive Explanation:
While Base R has `read.csv()`, modern data scientists use `read_csv()` from the `readr` package (part of the Tidyverse). It is significantly faster, does not automatically convert strings to factors, and imports the data directly as a clean 'tibble'. A CSV file is simply a text file where each value is separated by a comma, making it universally readable across all software.

Real-Life Analogy:
Importing a CSV is like receiving a long, continuous text message with words separated by commas, and R instantly builds an organized spreadsheet table out of it.

Syntax Structure:

```
library(readr)
my_data <- read_csv("file_path.csv")
```

R Code Example

```
library(readr)

# Simulating reading a CSV file from a local directory
# sales_data <- read_csv("data/sales_2024.csv")

# Using built-in readr sample data for demonstration
sales_data <- read_csv(readr_example("mtcars.csv"))
print(head(sales_data, 3))
```

Console Output

```
# A tibble: 3 × 11
    mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb

1  21       6   160   110  3.9   2.62  16.5     0     1     4     4
2  21       6   160   110  3.9   2.88  17.0     0     1     4     4
3  22.8     4   108    93  3.85  2.32  18.6     1     1     4     1
```

Line-by-Line Explanation:

We load the `readr` package. `read_csv()` locates the file, infers the data types of each column (e.g., `` for numbers), and imports it as a tibble. `head(..., 3)` previews the first 3 rows.

Real-World Use Case:

Every day, millions of CSV files containing daily stock prices, website traffic, or IoT sensor logs are imported into R using `read_csv()` for automated daily reporting.

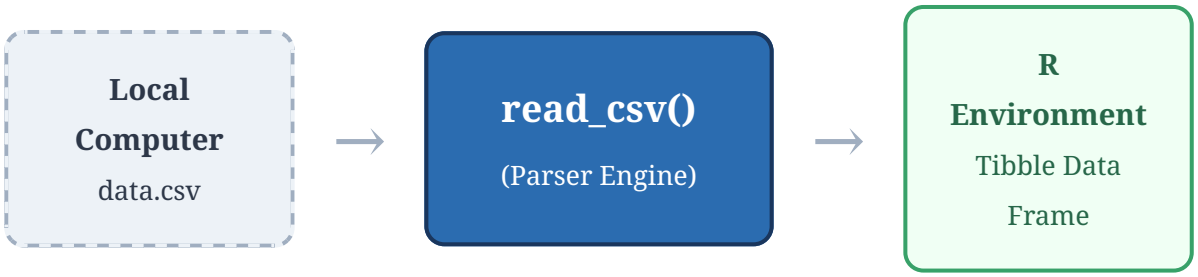
Common Mistakes:

Confusing `read.csv()` (Base R, returns a standard data.frame) with `read_csv()` (readr, returns a tibble). Using the wrong one can cause unexpected behavior with character strings.

Best Practices:
Always use `read_csv()` over `read.csv()`. Additionally, inspect the console message it prints upon loading to ensure it guessed your column types correctly.

Practice Challenge:
Use `read_csv()` to import any standard CSV file on your computer. Check its class using `class()`.

Data Import Workflow



Comparison: Base R vs readr

Feature	Base R `read.csv()`	Tidyverse `read_csv()`
Speed	Standard	**Extremely Fast (C++ backed)**
Output Object	Standard data.frame	Tibble (tbl_df)
String Conversion	Historically changed strings to factors.	Leaves strings alone.

2. Reading Excel Files (readxl)

CONCEPT:
Excel files (.xlsx) contain complex metadata and multiple sheets. The 'readxl' package extracts pure data from these spreadsheets.

Deep Dive Explanation:
Unlike CSVs, Excel files are proprietary Microsoft formats that contain colors, formulas, and multiple sheets. You cannot use `read_csv()` on an Excel file. Instead, the `readxl` package provides `read_excel()`. You can specify exactly which sheet to read and which rows/columns to skip if the Excel file has messy headers.

Real-Life Analogy:
A CSV is a plain text recipe. An Excel file is a glossy magazine page with photos, margins, and notes. `read_excel()` acts as a scanner that ignores the glossy formatting and just extracts the raw data table.

Syntax Structure:

```
library(readxl)
my_excel <- read_excel("file.xlsx", sheet = "Sheet1")
```

R Code Example

```
library(readxl)

# Finding a sample excel file included with the package
excel_file <- readxl_example("datasets.xlsx")

# Read the specific sheet named 'mtcars'
excel_data <- read_excel(excel_file, sheet = "mtcars")
print(head(excel_data, 2))
```

Console Output

```
# A tibble: 2 × 11
    mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb

1    21     6   160   110   3.9  2.62  16.5     0     1     4     4
2    21     6   160   110   3.9  2.88  17.0     0     1     4     4
```

Line-by-Line Explanation:

We specify the exact sheet we want to read using the `sheet` argument. R cleanly extracts the table, ignoring any Excel-specific formatting, and returns a tibble.

Real-World Use Case:

Corporate finance teams live in Excel. Data analysts use `read_excel()` to pull quarterly financial statements directly from network drives into R for automated auditing.

Common Mistakes:

Forgetting that `readxl` is NOT automatically loaded when you run `library(tidyverse)`. You must explicitly type `library(readxl)`.

Best Practices:

If an Excel file has a title or logo in the first 3 rows, use `read_excel("file.xlsx", skip = 3)` to skip the junk and start reading exactly where the column names begin.

Practice Challenge:

Look at the documentation `?read_excel`. How would you read only the first 50 rows of an Excel file?

3. Importing Text Files (read_delim)

CONCEPT:
For files separated by custom characters (like tabs, semicolons, or pipes), 'read_delim()' offers ultimate flexibility.

Deep Dive Explanation:
Not all data is comma-separated. Tab-Separated Values (.tsv) or Pipe-Separated Values (.txt) are common in database exports. The `read_delim()` function allows you to specify the exact 'delimiter' (the character separating the columns) using the `delim` argument.

Real-Life Analogy:
If `read_csv()` expects a comma as a brick wall between rooms, `read_delim()` lets you define what the wall is made of—whether it's a pipe `|`, a tab `\t`, or a semicolon `;`.

Syntax Structure:

```
read_delim("file.txt", delim = "|")
```

R Code Example

```
library(readr)

# Creating a temporary mock text file with pipe '|' delimiters
mock_text <- "Name|Age|Role\nAlice|30|Engineer\nBob|25|Analyst"

# Reading the custom delimited data
custom_data <- read_delim(mock_text, delim = "|", show_col_types =
FALSE)
print(custom_data)
```

Console Output

```
# A tibble: 2 × 3
  Name    Age Role

1 Alice    30 Engineer
2 Bob      25 Analyst
```

Line-by-Line Explanation:

We informed R that the delimiter separating the columns is the pipe character `|`. R successfully split the raw text string into three distinct columns: Name, Age, and Role.

Real-World Use Case:

Exporting data from legacy IBM mainframe databases often produces `.txt` files separated by pipes `|` or tildes `~` to avoid conflicts with commas inside text fields.

Common Mistakes:

Using `read_csv()` on a tab-separated file. R will fail to split the columns and load everything into a single, massive, unusable column.

Best Practices:

If you are unsure of the delimiter, open the `.txt` file in a basic text editor (like Notepad) first. Look at how the values are separated before choosing your R function.

Practice Challenge:

Write the command to read a file named `data.tsv` where columns are separated by tabs (`\t`).

File Formats: CSV vs Excel vs Text

Format	Extension	Separator	R Package / Function
CSV	.csv	Comma ,	`readr::read_csv()`
Excel	.xlsx / .xls	Binary Proprietary	`readxl::read_excel()`
Text / TSV	.txt / .tsv	Tabs `\t`, Pipes `|`, etc.	`readr::read_delim()`

4. Exporting Data (write_csv)

> **CONCEPT:**
> Exporting data saves your cleaned and manipulated R Data Frame back to your computer's hard drive.

Deep Dive Explanation:
After spending hours cleaning data, you need to save it so other people or programs can use it. The `write_csv()` function from the `readr` package converts your R tibble or data frame back into a standard `.csv` file. It is fast, and unlike Base R's `write.csv()`, it does not automatically inject annoying row numbers (1, 2, 3...) into your final file.

> **Real-Life Analogy:**
> Importing is bringing raw ingredients into the kitchen. Exporting is taking the finished, baked cake and boxing it up for the customer to take home.

Syntax Structure:

```
write_csv(my_dataframe, "path/to/save/clean_data.csv")
```

R Code Example

```
library(readr)

# Let's say we have a cleaned dataframe
cleaned_df <- data.frame(Name = c("Alice", "Bob"), Score = c(95,
80))

# Exporting the data to a CSV file
# write_csv(cleaned_df, "cleaned_scores.csv")

print("File successfully exported to working directory as
'cleaned_scores.csv'")
```

Console Output

```
[1] "File successfully exported to working directory as
'cleaned_scores.csv'"
```

Line-by-Line Explanation:

`write_csv()` takes two arguments: the name of the R object you want to save (`cleaned_df`), and the file path/name you want to create on your hard drive (`"cleaned_scores.csv"`).

Real-World Use Case:

A data engineer runs a nightly script that cleans raw sales data and exports it as `clean_sales.csv`, which a Tableau dashboard then reads the next morning.

Common Mistakes:

Forgetting the `.csv` extension in the file path. E.g., `write_csv(data, "clean_sales")`. The file will save, but your computer won't know it's a spreadsheet.

Best Practices:
Use `write_csv()` instead of base R's `write.csv()`. If you must use base R, always add `row.names = FALSE` so you don't ruin the dataset with an unwanted index column.

Practice Challenge:
Write the exact command to export a tibble called `inventory` to a file named `stock_2024.csv`.

5. Understanding and Detecting Missing Values (NA)

CONCEPT:
NA (Not Available) represents explicitly missing data. Detecting them is the first step of data cleaning.

Deep Dive Explanation:
In real-world data, fields are often left blank. R represents these blanks as `NA`. Because `NA` represents the 'unknown', any math performed on `NA` results in `NA`. To clean data, you must first find where the missing values are. We use `is.na()` to create a logical map of missingness, and combining it with `sum()` tells us exactly how many `NA` values exist in a column or dataset.

Real-Life Analogy:
An `NA` is a pothole in a road. You can't just drive over it normally. You must first scan the road (`is.na()`) to identify where the potholes are before deciding how to fix them.

Syntax Structure:

```
sum(is.na(dataset))
sum(is.na(dataset$column_name))
```

R Code Example

```
temperatures <- c(30, 32, NA, 29, NA, 35)

# Check which specific elements are NA
print(is.na(temperatures))

# Count total NA values in the vector
total_missing <- sum(is.na(temperatures))
print(paste("Total missing values:", total_missing))
```

Console Output

```
[1] FALSE FALSE  TRUE FALSE  TRUE FALSE
[1] "Total missing values: 2"
```

Line-by-Line Explanation:

`is.na()` returns `TRUE` wherever an `NA` exists. Because R treats `TRUE` as 1 and `FALSE` as 0, wrapping `sum()` around `is.na()` literally counts the number of `TRUE`s, instantly giving us the total count of missing values.

Real-World Use Case:

Before running a predictive algorithm, an analyst runs `sum(is.na(df))` on a massive dataset of 1 million rows to ensure no missing values will crash the model.

Common Mistakes:

Typing `temperatures == NA`. You cannot compare a value to an unknown. It will simply return a string of NAs. Always use `is.na()`.

Best Practices:
The very first command you should run after importing a new dataset is `summary(df)` or `colSums(is.na(df))` to map out the missing data landscape.

Practice Challenge:
Create a vector `v <- c(1, NA, 3, NA)`. Write code to count how many NAs are in it.

6. Handling Missing Values (tidyr)

CONCEPT:
Once NAs are found, you must either remove them (drop_na) or fill them in with an estimated value (replace_na).

Deep Dive Explanation:
The `tidyr` package provides clean solutions for missing data. 1. **Dropping**: `drop_na()` removes any entire row that contains an `NA`. Use this when you have millions of rows and losing a few doesn't matter. 2. **Imputing (Replacing)**: `replace_na()` fills the `NA` with a specific value (like 0, or the column's average). Use this when every row is critical.

Real-Life Analogy:
If a student's test score is missing (`NA`), you can either expel the student from the class record entirely (`drop_na`), or give them an automatic zero so they stay in the record (`replace_na`).

Syntax Structure:

```r
library(tidyr)
# Drops rows with NA
drop_na(dataset, column_name)

# Replaces NA with a specific value
replace_na(dataset, list(column_name = value))
```

R Code Example

```r
library(tidyr)
library(dplyr)

data <- tibble(Name = c("Ali", "Bob", "Cal"), Score = c(90, NA, 85))

# 1. Drop the row containing NA
clean_dropped <- drop_na(data, Score)
print(clean_dropped)

# 2. Replace NA with 0
clean_replaced <- replace_na(data, list(Score = 0))
print(clean_replaced)
```

Console Output

```
# A tibble: 2 × 2
  Name  Score

1 Ali      90
2 Cal      85

# A tibble: 3 × 2
  Name  Score

1 Ali      90
2 Bob       0
3 Cal      85
```

Line-by-Line Explanation:

Notice how `drop_na` completely deleted 'Bob' from the dataset because his score was missing. `replace_na` kept Bob, but substituted the `NA` in his Score column with a `0`.

Real-World Use Case:

In banking, missing 'Months as Customer' might be replaced with the median (Imputation), while missing 'Social Security Numbers' might mean the row must be deleted entirely (Dropping).

Common Mistakes:

Dropping NAs without realizing it deleted 80% of your dataset because one minor column was mostly blank. Always count NAs before dropping.

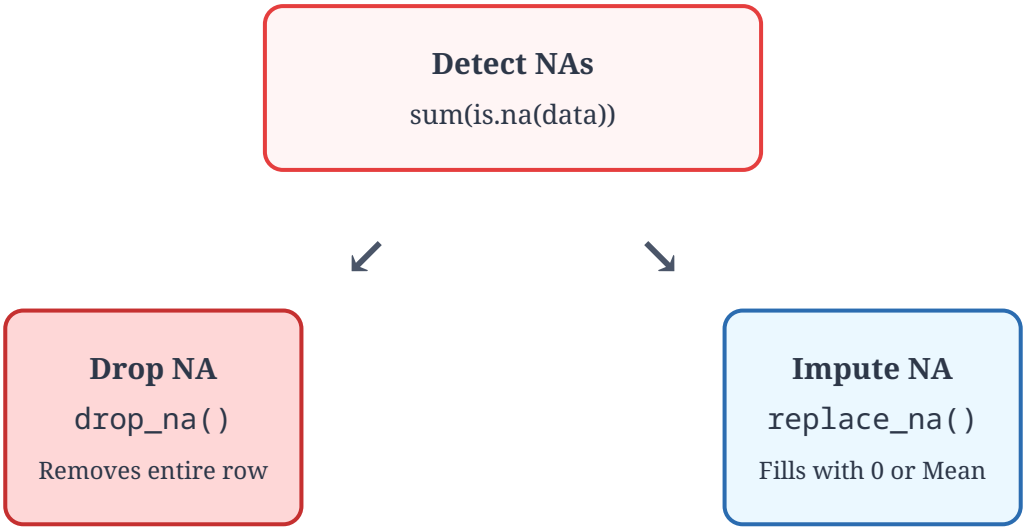
Best Practices:

Only use `drop_na()` on crucial target variables (like 'Sales'). Use `replace_na()` for supporting variables (like 'Discount_Applied' -> replace with 0).

Practice Challenge:

Use `replace_na` to change all NA values in a column named `age` to 25.

Missing-Data Workflow



Comparison: NA vs Blank

Data State	Code Representation	Meaning in R
Explicitly Missing	`NA`	An empty void. Fails math operations.
Empty Text	`""`	A valid character string with zero letters.
Null Object	`NULL`	The object itself does not exist in memory.

7. Removing Duplicates (dplyr)

CONCEPT:
The distinct() function easily isolates unique rows and removes exact duplicates from your dataset.

Deep Dive Explanation:
Due to system errors or human data entry, datasets often contain duplicate rows. In the `dplyr` package, the `distinct()` function scans the dataset. If it finds two rows that are perfectly identical across all columns, it deletes the duplicates and keeps only the first unique instance. You can also ask it to find uniqueness based on just one specific column.

Real-Life Analogy:
If a guest accidentally RSVPs to your wedding twice with the exact same name and meal choice, `distinct()` removes the second RSVP so you don't buy an extra dinner.

Syntax Structure:

```
library(dplyr)
distinct(dataset)              # Uses all columns
distinct(dataset, column_name, .keep_all = TRUE)
```

R Code Example

```
library(dplyr)

# Dataset with a duplicate row for 'Ali'
attendees <- tibble(
  ID = c(1, 2, 1, 3),
  Name = c("Ali", "Bob", "Ali", "Cat")
)

# Remove identical duplicate rows
unique_attendees <- distinct(attendees)
print(unique_attendees)
```

Console Output

```
# A tibble: 3 × 2
     ID Name

1     1 Ali
2     2 Bob
3     3 Cat
```

Line-by-Line Explanation:

Row 1 and Row 3 were identical (ID=1, Name=Ali). The `distinct()` function recognized this and completely dropped Row 3, returning only the unique rows.

Real-World Use Case:

E-commerce databases often log a transaction twice if a user double-clicks the 'Checkout' button. `distinct(Order_ID)` is run to clean the database before calculating revenue.

Common Mistakes:

Running `distinct(Column_A)` without adding `.keep_all = TRUE`. If you don't add `.keep_all = TRUE`, R will delete every other column in the dataset and return a single column containing only the unique values of Column A.

Best Practices:
Always run a row count (`nrow()`) before and after running `distinct()` to see exactly how many duplicate rows were hiding in your data.

Practice Challenge:
Write the `dplyr` command to keep only unique values based on the `Email` column, while keeping all other columns intact.

8. Renaming Columns (dplyr)

CONCEPT:
The rename() function allows you to safely change column headers to make them clean, readable, and standard.

Deep Dive Explanation:
Raw data often comes with terrible column headers containing spaces, symbols, or confusing abbreviations (e.g., `Q1_REV_$$`). The `rename()` function from `dplyr` fixes this securely. The syntax is always `rename(data, NEW_NAME = OLD_NAME)`.

Real-Life Analogy:
Renaming a column is simply peeling off the old, messy label on a moving box and sticking on a fresh, clearly printed label. The contents inside the box remain untouched.

Syntax Structure:

```
rename(dataset, new_name = old_name)
```

R Code Example

```
library(dplyr)

messy_data <- tibble(
  `Emp ID#` = c(101, 102),
  `SALARY_2024` = c(50000, 60000)
)

# Rename columns to clean, snake_case formats
clean_data <- rename(messy_data,
                     emp_id = `Emp ID#`,
                     salary = `SALARY_2024`)

print(clean_data)
```

Console Output

```
# A tibble: 2 × 2
  emp_id salary

1    101  50000
2    102  60000
```

Line-by-Line Explanation:

Notice how the original messy column `Emp ID#` required backticks (``) because it contained a space and a symbol. We successfully renamed it to a safe, programmatic name: `emp_id`.

Real-World Use Case:

Data imported from Google Analytics often has columns like `Session Duration (Seconds)`. Analysts rename this to `session_duration_sec` to make writing code later much easier.

Common Mistakes:

Putting the old name first! Many beginners type `rename(OLD = NEW)`, which throws an error. It is strictly `NEW = OLD`.

Best Practices:
Standardize all column names to lowercase and use underscores instead of spaces (`snake_case`). E.g., `first_name`, `total_sales`.

Practice Challenge:
Write the code to rename a column currently called `Phone#` to `phone_number`.

9. Selecting Columns (dplyr)

CONCEPT:
The select() function isolates specific columns (variables) you want to keep, or drops columns you don't need.

Deep Dive Explanation:
Datasets can have hundreds of columns, but you might only need three for your analysis. The `select()` function slices the dataset **vertically**. You can list the columns you want to keep, or you can use a minus sign `-` to list the columns you want to drop. It is a core part of the `dplyr` 'grammar of data manipulation'.

Real-Life Analogy:
If your dataset is a massive restaurant menu, `select()` is choosing to look only at the 'Desserts' and 'Beverages' sections, completely ignoring the 'Appetizers' section.

Syntax Structure:

```
select(dataset, col1, col2)       # Keep columns
select(dataset, -col1)            # Drop column
```

R Code Example

```
library(dplyr)

staff <- tibble(
  ID = 1:2,
  Name = c("Dithsa", "Mithila"),
  SSN = c("xxx-xx-xxxx", "yyy-yy-yyyy"),
  Department = c("IT", "HR")
)

# Select exactly what we want, dropping highly sensitive SSN
safe_staff <- select(staff, ID, Name, Department)

# Alternatively, just drop the one we don't want using minus
# safe_staff <- select(staff, -SSN)

print(safe_staff)
```

Console Output

```
# A tibble: 2 × 3
     ID Name    Department

1     1 Dithsa  IT
2     2 Mithila HR
```

Line-by-Line Explanation:

The original dataset had 4 columns. By feeding `ID`, `Name`, and `Department` into `select()`, R returned a smaller dataset omitting the `SSN` column.

Real-World Use Case:

When building a Machine Learning model to predict housing prices, an analyst will use `select()` to drop the `Homeowner_Name` column, as the name has no mathematical impact on the price.

Common Mistakes:
Confusing `select()` with `filter()`. `select()` works strictly on Columns (Vertical). `filter()` works strictly on Rows (Horizontal).

Best Practices:
If a dataset has 50 columns and you want to drop 1, use `select(data, -bad_col)`. If you want to keep 3, use `select(data, col1, col2, col3)`. Use whichever method requires less typing.

Practice Challenge:
Use `select()` to drop a column named `password_hash` from a dataframe named `users`.

Comparison: select() vs filter()

Function	Action	Direction	Example Use Case
select()	Chooses Variables	Vertical (Columns)	Dropping SSN and Phone Number columns.
filter()	Chooses Observations	Horizontal (Rows)	Keeping only customers from New York.

10. Filtering Rows (dplyr)

CONCEPT:
The filter() function isolates specific rows (observations) based on logical conditions.

Deep Dive Explanation:
While `select()` slices vertically, `filter()` slices **horizontally**. You provide `filter()` with a logical test (e.g., `Age >= 18`). It evaluates every single row in the dataset. If a row evaluates to TRUE, it keeps it. If it evaluates to FALSE, it drops it. You can combine multiple conditions using `&` (AND) and `|` (OR).

Real-Life Analogy:
If a dataset is a crowd of people, `filter()` is shouting: 'Everyone wearing a blue shirt, step forward! Everyone else, leave.'

Syntax Structure:

```
filter(dataset, condition1, condition2)
```

R Code Example

```
library(dplyr)

inventory <- tibble(
  Item = c("Laptop", "Mouse", "Keyboard", "Monitor"),
  Price = c(1200, 25, 45, 300),
  In_Stock = c(TRUE, TRUE, FALSE, TRUE)
)

# Filter for items > $50 AND currently in stock
premium_available <- filter(inventory, Price > 50 & In_Stock ==
TRUE)

print(premium_available)
```

Console Output

```
# A tibble: 2 × 3
  Item    Price In_Stock

1 Laptop   1200 TRUE
2 Monitor   300 TRUE
```

Line-by-Line Explanation:

The Mouse was dropped because its price was not > 50. The Keyboard was dropped because `In_Stock` was FALSE. Only the Laptop and Monitor met BOTH strict conditions.

Real-World Use Case:

A marketing team wants to run a campaign. They run `filter(customers, Last_Purchase_Days < 30 & Subscribed_Newsletter == TRUE)` to isolate highly engaged users.

Common Mistakes:

Using a single `=` instead of `==` for exact matching. `filter(data, City = "London")` will crash. It MUST be `filter(data, City == "London")`.

Best Practices:
When filtering for text strings, remember that R is strictly case-sensitive. E.g., `"london"` does not equal `"London"`.

Practice Challenge:
Write a `filter()` command to find all rows where `Age` is greater than or equal to 65.

11. Sorting Data (arrange)

CONCEPT:
The arrange() function orders the rows of a dataset sequentially based on the values of one or more columns.

Deep Dive Explanation:
Data naturally arrives disorganized. To make sense of it, we use `arrange()` to sort rows chronologically, alphabetically, or numerically. By default, `arrange()` sorts in Ascending order (A to Z, 1 to 10). To sort in Descending order (Z to A, 10 to 1), you wrap the column name in the `desc()` function.

Real-Life Analogy:
If `filter` decides who gets into the concert, `arrange` tells them exactly what order to stand in line based on their ticket number.

Syntax Structure:

```
arrange(dataset, column_name)          # Ascending
arrange(dataset, desc(column_name))    # Descending
```

R Code Example

```
library(dplyr)

scores <- tibble(
  Name = c("Dithsa", "Ali", "Mithila"),
  Marks = c(85, 92, 95)
)

# Sort ascending alphabetically by Name
alpha_sort <- arrange(scores, Name)

# Sort descending numerically by Marks (Highest first)
top_scores <- arrange(scores, desc(Marks))

print(top_scores)
```

Console Output

```
# A tibble: 3 × 2
  Name    Marks

1 Mithila    95
2 Ali        92
3 Dithsa     85
```

Line-by-Line Explanation:

Wrapping `desc()` around `Marks` forced R to place the highest number (95) at row 1. Notice how the `Name` column perfectly followed the `Marks` sorting, keeping the student records intact.

Real-World Use Case:

An HR department uses `arrange(employees, desc(Hire_Date))` to view the most recently hired employees at the very top of their report.

Common Mistakes:

Forgetting that `arrange()` creates a temporary sorted view. If you don't assign it to a variable (e.g., `sorted_data <- arrange(...)`), the dataset remains permanently unsorted in memory.

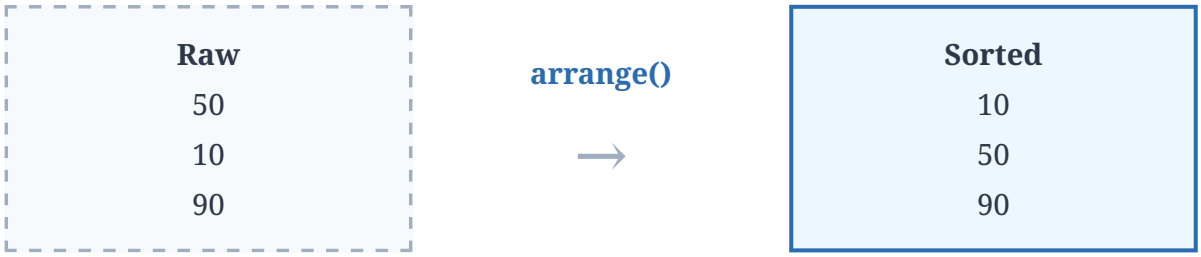
Best Practices:

You can sort by multiple columns to break ties. E.g., `arrange(data, Department, desc(Salary))` sorts by Department alphabetically, and within each department, sorts by highest salary.

Practice Challenge:

Write the command to sort a dataframe `sales` by `Revenue` from lowest to highest.

Sorting Workflow



To flip the order to 90, 50, 10, use `arrange(desc(column))`.

12. Creating New Columns (mutate)

CONCEPT:

The mutate() function creates brand new columns or modifies existing ones while keeping the dataset intact.

Deep Dive Explanation:

Data analysis requires calculating new metrics (like calculating Tax based on a Price column). `mutate()` allows you to attach new columns to the end of your dataset. The new column is created using vectorized mathematical formulas based on existing columns. Unlike base R (`df$new_col <- ...`), `mutate()` allows you to create multiple complex columns in a single, readable step.

> **Real-Life Analogy:**
> If your dataset is a house, `mutate()` is building a new extension room onto the back of the house using building materials (data) from the existing rooms.

Syntax Structure:

```
mutate(dataset, new_col_name = math_logic)
```

R Code Example

```
library(dplyr)

cart <- tibble(
  Item = c("Book", "Pen"),
  Price = c(20, 5)
)

# Create a Tax column (10%) and a Total column
cart_updated <- mutate(cart,
                       Tax = Price * 0.10,
                       Total = Price + Tax)

print(cart_updated)
```

Console Output

```
# A tibble: 2 × 4
  Item  Price   Tax Total

1 Book     20   2.0  22.0
2 Pen       5   0.5   5.5
```

Line-by-Line Explanation:

Inside `mutate()`, we created `Tax` by multiplying `Price` by 10%. Immediately in the next line, we used that freshly created `Tax` column to generate the `Total` column. This sequential awareness makes `mutate()` incredibly powerful.

Real-World Use Case:

Creating a Profit column by subtracting `Cost_of_Goods` from `Gross_Revenue` for every row in a database of a million transactions.

Common Mistakes:

Using `==` instead of `=` inside mutate. `mutate(new_col == Price * 2)` will fail. You are assigning a name, so you must use the single equals sign.

Best Practices:

Combine `mutate()` with `ifelse()` to create powerful categorical columns. E.g., `mutate(data, Status = ifelse(Score > 50, "Pass", "Fail"))`.

Practice Challenge:

Write a `mutate` command to create a column `Age_in_Months` by multiplying an existing `Age` column by 12.

13. Basic Data Cleaning & The Pipe

CONCEPT:
The Pipe operator (%>%) connects multiple cleaning steps into a single, flowing, easily readable pipeline.

Deep Dive Explanation:
In classic R, running `filter()`, `select()`, and `mutate()` required saving the data to temporary variables at each step, creating messy, unreadable code. The **Pipe operator (`%>%`)** solves this. It takes the dataset on its left and "pipes" it directly into the first argument of the function on its right. This allows you to chain a dozen cleaning functions sequentially, reading like a recipe: 'Take data, AND THEN filter, AND THEN mutate, AND THEN arrange.'

Real-Life Analogy:
The pipe `%>%` is literally a plumbing pipe connecting water filtration tanks. The dirty water (raw data) enters Tank 1 (filter). It pipes directly into Tank 2 (mutate), and flows out as clean, drinkable water (final data frame).

Syntax Structure:

```
clean_data <- raw_data %>%
  filter(...) %>%
  select(...) %>%
  mutate(...)
```

R Code Example

```
library(dplyr)

raw_data <- tibble(
  Name = c("Ali", "Bob", "Cat"),
  Age = c(30, 15, 25),
  Dept = c("IT", "HR", "IT")
)

# Chaining commands with the pipe %>%
clean_data <- raw_data %>%
  filter(Age >= 18) %>%      # Step 1: Keep adults
  select(Name, Dept) %>%     # Step 2: Drop Age column
  arrange(Name)              # Step 3: Sort alphabetically

print(clean_data)
```

Console Output

```
# A tibble: 2 × 2
  Name  Dept

1 Ali   IT
2 Cat   IT
```

Line-by-Line Explanation:

Notice we didn't have to write the dataset name inside `filter`, `select`, or `arrange`. The pipe `%>%` automatically shoved `raw_data` into the first invisible slot of `filter()`, and piped the result down the chain.

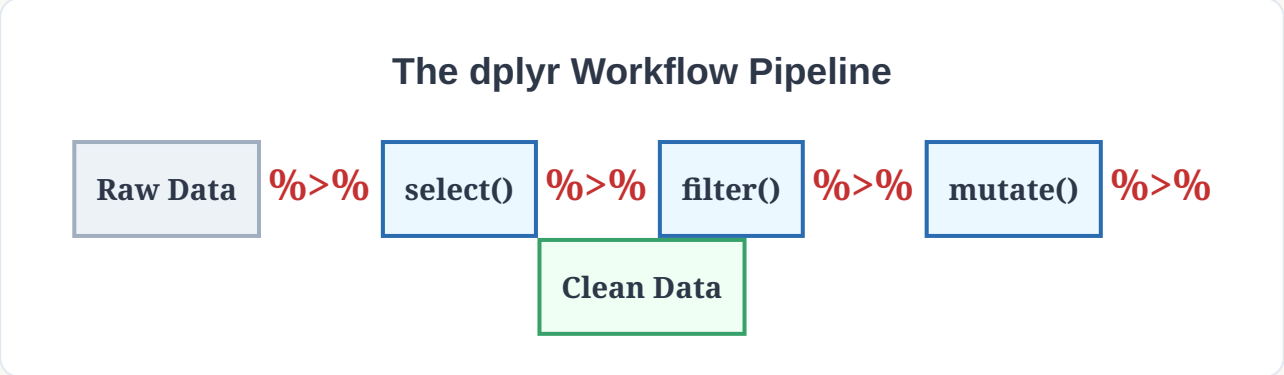
Real-World Use Case:

Every modern R script utilizes the `%>%` (or the newer base R `|>`) pipe to construct data pipelines that clean millions of rows of data sequentially without cluttering the computer's memory.

Common Mistakes:
Putting the pipe `%>%` at the *beginning* of the next line. It must ALWAYS be at the *end* of the line so R knows the command isn't finished yet.

Best Practices:
Always use the pipe to connect `dplyr` verbs. It changes R code from an 'inside-out' mathematical puzzle into standard left-to-right English.

Practice Challenge:
Take a dataset `df`, pipe it into `filter(Score > 50)`, and pipe that into `arrange(Score)`.



14. The Concept of Tidy Data

CONCEPT:
Tidy Data is a universally recognized standard for formatting data structurally to make analysis seamless.

Deep Dive Explanation:
Developed by Hadley Wickham, 'Tidy Data' is the philosophy behind the entire Tidyverse. It has three unbreakable rules: 1. Every **variable** must be its own **column**. 2. Every **observation** must be its own **row**. 3. Every **value** must have its own **cell**. If

your data violates these rules (e.g., column names are months like Jan, Feb, Mar instead of a single 'Month' column), standard R functions and charting tools will struggle to process it.

> **Real-Life Analogy:**
> Tidy data is an organized toolbox where every wrench size has exactly one specific slot. Untidy data is throwing all wrenches, screws, and hammers into a single bucket and trying to find the 10mm socket in the dark.

Syntax Structure:

```
No specific syntax; it is a structural philosophy applied during data cleaning.
```

R Code Example

```
# UNTIDY DATA (Columns represent values, not variables)
untidy <- tibble(
  Patient = c("John", "Mary"),
  `Trial 1` = c(120, 110),
  `Trial 2` = c(115, 112)
)

# TIDY DATA (Variables are strictly columns: Patient, Trial, HeartRate)
tidy <- tibble(
  Patient = c("John", "John", "Mary", "Mary"),
  Trial = c("Trial 1", "Trial 2", "Trial 1", "Trial 2"),
  HeartRate = c(120, 115, 110, 112)
)
print("Tidy data adheres to strict tabular rules.")
```

Console Output

```
[1] "Tidy data adheres to strict tabular rules."
```

Line-by-Line Explanation:

In the untidy version, 'Trial 1' and 'Trial 2' are columns, but 'Trial' is actually a variable! The tidy version collapses them so that 'Trial' is a column, and the actual metric 'HeartRate' has its own dedicated column.

Real-World Use Case:

Before plotting a line chart in `ggplot2`, data must be reshaped into Tidy format so the plotting engine knows exactly which column represents the X-axis and which is the Y-axis.

Common Mistakes:

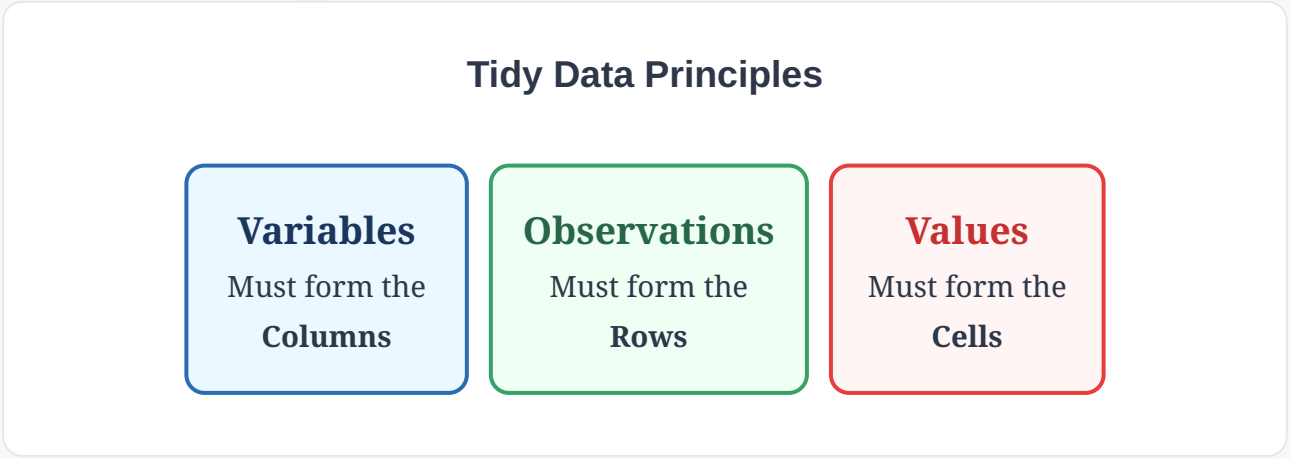
Trying to run complex predictive models or charts on messy, untidy data (like cross-tabulations or pivot tables). Base models require strictly Tidy structures.

Best Practices:

The `pivot_longer()` function from `tidyr` is used to convert untidy 'wide' data (like our untidy example) into perfectly Tidy 'long' data.

Practice Challenge:

Look at a standard spreadsheet where columns are years (2020, 2021, 2022). Identify why this violates Tidy Data Rule 1.



HANDS-ON PROJECT

IT STUDY HUB

REAL-WORLD DATA CLEANING PROJECT

Problem Statement:
You are a Data Analyst for a corporate HR department. You receive a monthly simulated CSV file containing raw employee data. It is plagued with missing values, duplicate entries, terrible column names, and sensitive columns that need to be dropped. Your goal is to write a single `dplyr` pipeline to ingest the data, surgically clean it, create a new calculated 'Bonus' column, sort it, and export the pristine dataset for the Tableau team.

1. Simulating the Messy Data (Setup)

```
library(tibble)
library(readr)

# Creating a messy dataset and saving it to disk to simulate
reality
messy_data <- tibble(
  `Emp ID` = c(101, 102, 103, 101, 104, 105),
  `Full Name` = c("Alice Smith", "Bob Jones", "Charlie Day",
"Alice Smith", NA, "Eva Green"),
  Department = c("Sales", "IT", "Sales", "Sales", "HR", "IT"),
  `Base_Salary_$$` = c(60000, 75000, NA, 60000, 50000, 80000),
  SSN = c("xxx-xx-xxxx", "yyy-yy-yyyy", "zzz-zz-zzzz", "xxx-xx-
xxxx", "aaa-aa-aaaa", "bbb-bb-bbbb")
)

# Export as messy CSV
write_csv(messy_data, "messy_employees.csv")
```

2. The Complete Data Cleaning Pipeline

```
library(dplyr)
library(tidyr)
library(readr)

# Step 1: Import the raw data
raw_employees <- read_csv("messy_employees.csv", show_col_types
= FALSE)

# Step 2: Build the Tidy Pipeline
clean_employees <- raw_employees %>%

  # a. Remove exact duplicate rows (Fixes the double 'Alice
Smith' issue)
  distinct() %>%

  # b. Rename terrible column headers to snake_case
  rename(
    emp_id = `Emp ID`,
    full_name = `Full Name`,
    salary = `Base_Salary_$$`
  ) %>%

  # c. Select columns (Drop the highly sensitive SSN column)
  select(-SSN) %>%

  # d. Handle Missing Data
  # Drop row if 'full_name' is missing entirely
  drop_na(full_name) %>%
  # If salary is missing, assume a base of 45000
  replace_na(list(salary = 45000)) %>%

  # e. Filter data (Only keep employees making >= 50k)
  filter(salary >= 50000) %>%

  # f. Create a new calculated column (10% standard bonus)
  mutate(bonus_amount = salary * 0.10) %>%

  # g. Sort the final data by highest salary first
```

```
  arrange(desc(salary))

# Step 3: Export the pristine data
write_csv(clean_employees, "clean_employees.csv")

# Step 4: Inspect the final product
print(clean_employees)
```

Expected Console Output

```
# A tibble: 4 × 4
  emp_id full_name    Department salary bonus_amount

1    105 Eva Green    IT          80000         8000
2    102 Bob Jones    IT          75000         7500
3    101 Alice Smith  Sales       60000         6000
4    103 Charlie Day  Sales       45000         4500
```

Step-by-Step Explanation:

Our raw data had 6 rows and 5 columns. First, `distinct()` deleted the exact duplicate of Alice, leaving 5 rows. We safely renamed the columns so we could call them without backticks. `select(-SSN)` surgically removed the SSN column. We used `drop_na()` to delete Employee 104 because their name was `NA`. Charlie's missing salary was imputed with 45000 via `replace_na()`. We generated a `bonus_amount` using `mutate()`, sorted the whole thing, and dumped it back out to the hard drive. 6 messy rows became 4 pristine rows in one uninterrupted, highly readable pipe block.

Cleaning Checklist & Extension Challenge:

1. Always run `summary(data)` before cleaning to spot NAs.
2. Always drop duplicates *before* generating new unique IDs or row numbers.

Challenge: Modify the pipeline to create an additional logical column called `Highly_Paid` that equals TRUE if salary is > 70000, and FALSE otherwise.

ASSESSMENTS & INTERVIEW PREP

15+ Practice Questions

Test your fundamental knowledge of data cleaning in R.

1. What is the modern, fast function for reading CSV files found in the `readr` package?
2. If you want to read an Excel file, which specific package must you load first?
3. What argument inside `read_delim()` specifies how the text is separated?
4. Which function do you use to save an R dataframe back to your computer as a CSV?
5. Write the code to find out exactly how many NA values exist in an entire dataset.
6. What is the difference between `drop_na()` and `replace_na()`?
7. If a dataset has identical, repeated rows, what `dplyr` function removes them?
8. When renaming a column, what goes on the left side of the equals sign: the NEW name or the OLD name?
9. Write a `select()` command to drop a column called `Passwords`.
10. Does the `filter()` function apply to Rows or Columns?
11. How do you sort a dataframe alphabetically by the `Name` column?
12. If you want to sort a dataframe by the `Price` column from Highest to Lowest, what wrapper function must you add?
13. What function creates brand new calculated columns?
14. What is the symbol for the pipe operator, and what is its primary purpose?
15. Name one of the three unbreakable rules of 'Tidy Data'.
16. Why shouldn't you write `data == NA` to check for missing values?

IT STUDY HUB

20+ Viva Questions (With Answers)

These questions are designed for rapid-fire university viva or oral examinations.

Q1: What does CSV stand for?

Answer: Comma-Separated Values.

Q2: What is the main advantage of `read_csv()` over Base R's `read.csv()`?

Answer: It is significantly faster, does not convert strings to factors, and imports as a tibble.

Q3: Can `read_csv()` open `.xlsx` files?

Answer: No, CSV readers only read plain text. You must use `readxl::read_excel()` for Excel files.

Q4: What does the `skip` argument do in data import functions?

Answer: It skips a specified number of messy or blank rows at the very top of the file before reading the headers.

Q5: What is the difference between NA and NULL?

Answer: NA means explicitly missing data (a blank cell). NULL means the object itself doesn't exist.

Q6: How do you identify NAs computationally?

Answer: By using the `is.na()` function, often combined with `sum()`.

Q7: When should you use Imputation over Dropping?

Answer: When your dataset is small and deleting entire rows for one missing cell would destroy too much valuable data.

Q8: What does the `distinct()` function do?

Answer: It removes duplicate rows, keeping only the first unique instance.

Q9: What is the syntax rule for `rename()`?

Answer: `rename(data, new_name = old_name)`.

Q10: What is the difference between `select()` and `filter()`?

Answer: `select()` slices columns vertically. `filter()` slices rows horizontally based on logic.

IT STUDY HUB

Q11: Can `select()` be used to remove columns?

Answer: Yes, by adding a minus sign `-` in front of the column name.

Q12: How does `arrange()` sort by default?

Answer: In ascending order (A to Z, or Lowest to Highest).

Q13: How do you reverse the `arrange()` sorting order?

Answer: By wrapping the target column in the `desc()` function.

Q14: What is the purpose of `mutate()`?

Answer: To create new columns or alter existing ones based on mathematical formulas.

Q15: What is the keyboard shortcut/symbol for the pipe operator?

Answer: The symbol is `%>%` (or modern base R `|>`).

Q16: How does the pipe operator `%>%` work?

Answer: It takes the output of the code on its left and passes it directly into the first argument of the function on its right.

Q17: What are the main rules of Tidy Data?

Answer: 1. Variables are columns. 2. Observations are rows. 3. Values are cells.

Q18: What is 'Data Wrangling'?

Answer: The process of gathering, importing, cleaning, and transforming raw data into a usable format.

Q19: If column names have spaces in them, how do you refer to them in code?

Answer: You must wrap the column name in backticks (`).

Q20: Why do we use `write_csv()` instead of base R's `write.csv()`?

Answer: `write_csv()` is faster and prevents the automatic creation of a useless row-number index column in the exported file.

15+ Corporate Interview Questions

Prepare for data analyst technical rounds with these comprehensive data wrangling questions.

IT STUDY HUB

1. Explain the philosophical concept behind 'Tidy Data'. Why do `ggplot2` and machine learning libraries practically require it?

2. Walk me through your mental checklist when receiving a brand new, undocumented raw dataset.

3. Explain the difference between `read.csv()` and `read_csv()`. Why did the `tidyverse` team build the latter?

4. How does the pipe operator `%>%` improve the readability of data cleaning scripts compared to nested Base R code?

5. Your dataset has a column called `Phone Number` and you attempt to run `select(Phone Number)`. R throws an error. Why? How do you fix it?

6. If you run `drop_na()` on a dataset of 10,000 rows and end up with 500 rows, what architectural mistake did you likely make?

7. Explain the difference between `select()` and `filter()`. Provide a real-world example of when you would use them sequentially.

8. How would you use `mutate()` and `ifelse()` together to create a new categorical column based on a continuous numerical column?

9. What does the `.keep_all = TRUE` argument do inside the `distinct()` function, and why is it crucial?

10. You need to sort a dataframe by `Department` alphabetically, but within each department, sort by `Salary` highest to lowest. How do you write the `arrange()` code?

11. Describe a scenario where you would use `replace_na()` to impute a value rather than using `drop_na()`.

12. Why do we explicitly avoid using base R's `write.csv()` in automated production pipelines?

13. What is the difference between an explicitly missing `NA` value and an implicitly missing value in a database?

14. If you have an Excel file with multiple sheets, how does `read_excel()` handle this by default, and how do you target a specific sheet?

IT STUDY HUB

15. Explain how you would safely handle a text file imported via `read_delim()` where the delimiter is an unusual character, like a tilde (~).

IT STUDY HUB

MODULE WRAP-UP

Module Summary

In Module 5, you evolved from a programmer into a Data Analyst. You learned how to bridge R to the outside world, pulling raw data from CSVs, Excel files, and custom text formats, and successfully exporting it back out. More importantly, you learned that raw data is rarely perfect. Using the `dplyr` and `tidyr` packages, you constructed robust, logical pipelines connected by the `%>%` pipe. You mastered detecting and neutralizing `NA` values, deleting duplicates, selecting features, filtering populations, sorting data, and engineering brand new metrics using `mutate()`. Finally, you learned the absolute gold standard of data architecture: The Tidy Data philosophy. Your data is now clean, structured, and primed for heavy analysis.

IT STUDY HUB

Key Terms Dictionary

Term	Definition
CSV	Comma-Separated Values. The universal plain-text data format.
readr / readxl	Tidyverse packages engineered for high-speed file importation.
Pipeline / Pipe (%>%)	A chain of commands where data flows from one function into the next.
NA	Not Available. R's explicit marker for missing data points.
Imputation	The data science practice of intelligently guessing/replacing missing values.
select()	dplyr verb used to keep or drop Columns (Vertical subsetting).
filter()	dplyr verb used to keep or drop Rows (Horizontal subsetting).
mutate()	dplyr verb used to create new columns via mathematical formulas.
Tidy Data	A strict format where columns = variables, and rows = observations.

Quick Revision Checklist

- ✅ Do you know how to map missing values using `sum(is.na(data))`?
- ✅ Can you confidently choose between `drop_na()` and `replace_na()`?
- ✅ Do you understand the difference between `select()` (cols) and `filter()` (rows)?
- ✅ Do you know to use backticks (`) for column names containing spaces?
- ✅ Can you write a 3-step `%>%` pipeline mentally?

Data-Cleaning Coding Challenges

Challenge 1: The Cleaner
Create a tibble with 3 columns. Make sure row 2 is an exact duplicate of row 1. Make sure row 3 has an NA. Pipe this tibble into `distinct()`, and then into `drop_na()`. Print the result.

Challenge 2: The Accountant
Use `mutate()` on a mock dataset to calculate `Net_Income` by subtracting a `Taxes` column from a `Gross_Revenue` column.

Challenge 3: The Organizer
Write a `filter()` command that isolates employees in the "IT" department, then pipe it into an `arrange()` command to sort them by `desc(Salary)`.

WHAT COMES NEXT:

MODULE 6 — DATA VISUALIZATION WITH R

You now possess pristine, perfectly structured data. Next, we will introduce the world-renowned `ggplot2` package, teaching you how to translate your clean tables into stunning, publication-ready charts and graphs.